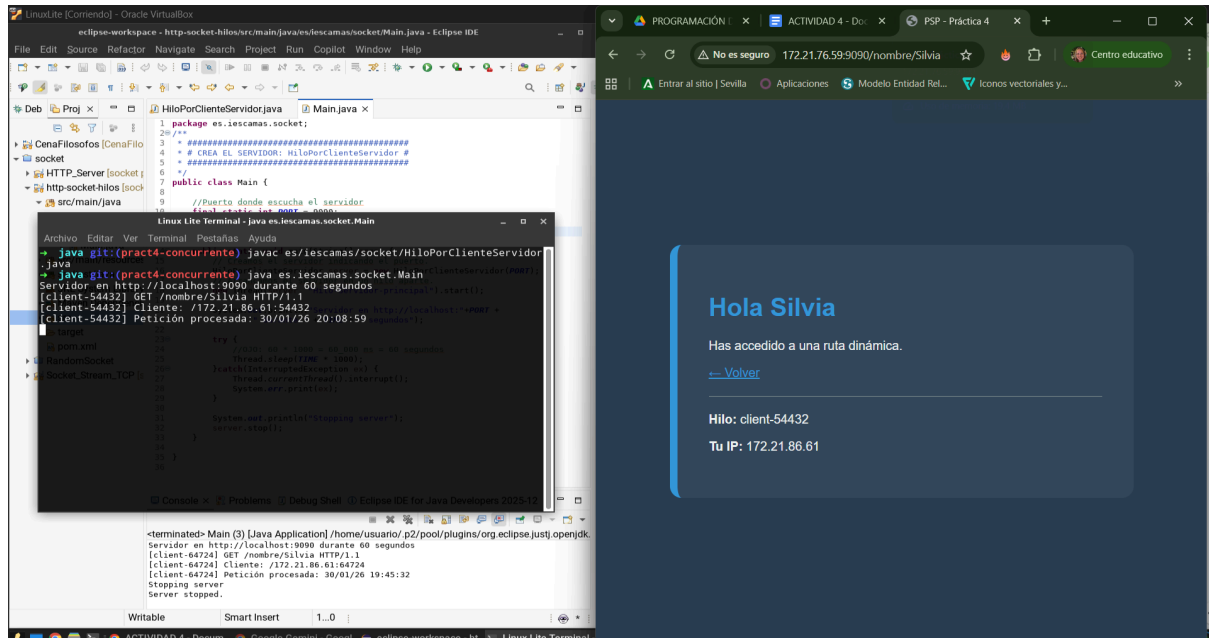


ACTIVIDAD 4

Mejora 1: Ruta dinámica /nombre/<nombre>



1. Qué se cambió

- **Servidor (HiloPorClienteServidor.java)** → Se añadió una estructura condicional `if (path.startsWith("/nombre/"))` para interceptar peticiones específicas. Si la ruta coincide, se utiliza el método `substring(8)` para extraer el texto que el usuario escribe después de la barra.
- **Respuesta** → En lugar de un mensaje estático, se creó la variable `mensajeSaludo` que inserta el nombre extraído directamente en la etiqueta `<h1>` del código HTML resultante.

2. Por qué se cambió

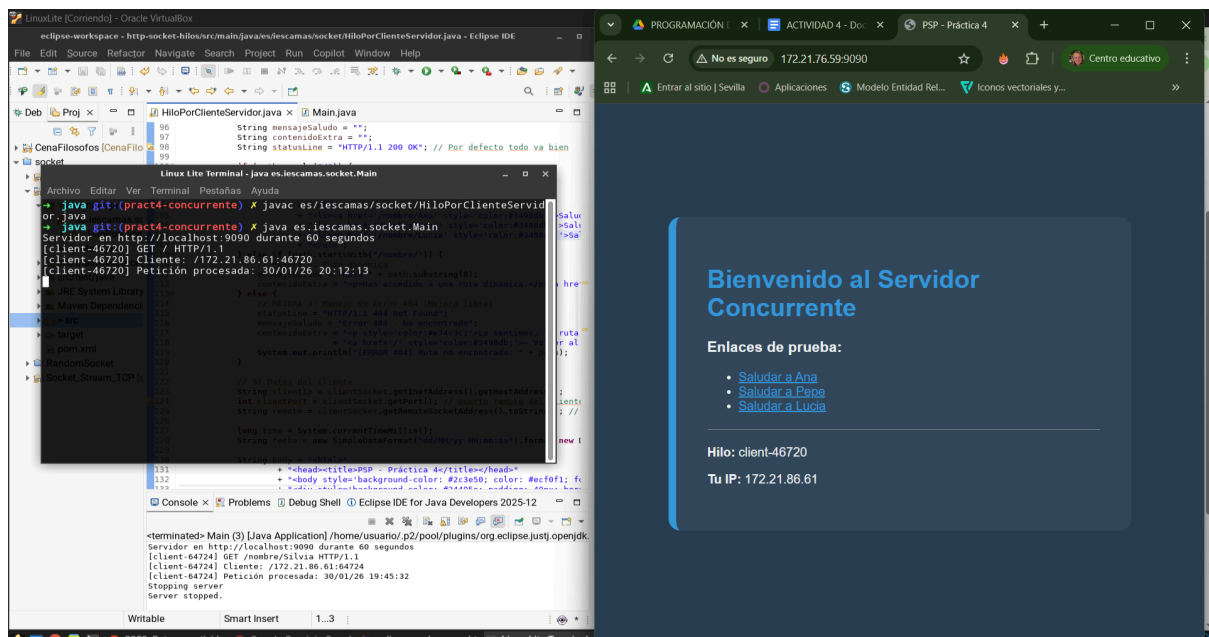
- **Procesamiento de Parámetros** → Se implementó para demostrar la capacidad del servidor de procesar datos variables enviados a través de la URL. Esto permite que una misma lógica de programación pueda ofrecer respuestas personalizadas a diferentes usuarios sin necesidad de crear archivos HTML individuales para cada nombre.

- **Dinamismo del Servicio** → Mejora la funcionalidad del servidor, pasando de ser un simple emisor de documentos fijos a un servicio capaz de interactuar con la entrada del cliente en tiempo real.

3. Cómo comprobar que funciona

1. **Lanzamiento** → Se arranca el servidor desde la terminal de la VM Linux ejecutando la clase Main. El servidor quedará a la escucha en el puerto 9090 durante 60 segundos.
2. **Petición del Cliente** → Desde el navegador en Windows (Host), introduce la dirección IP de la VM seguida de la ruta personalizada, por ejemplo: `http://172.21.76.59:9090/nombre/Silvia`.
3. **Resultado esperado** → El navegador debe mostrar una página donde el título principal sea "Hola Silvia".
4. **Verificación** → En la terminal de la VM Linux, debe aparecer el registro confirmando la ruta accedida:
 - a. `[client-48933] GET / nombre/Silvia HTTP/1.1`
 - b. `[client-48933] Cliente: / 172.21.86.61:48933`
 - c. `[client-48933] Petición procesada: 30/01/26 19:46:40`

Mejora 2: Página de Inicio (/) y Diseño Visual



1. Qué se cambió

- **Servidor (HiloPorClienteServidor.java)** → Se definió la ruta raíz (/) como un punto de entrada específico que carga un panel de bienvenida con enlaces directos (<a>) hacia diferentes nombres.

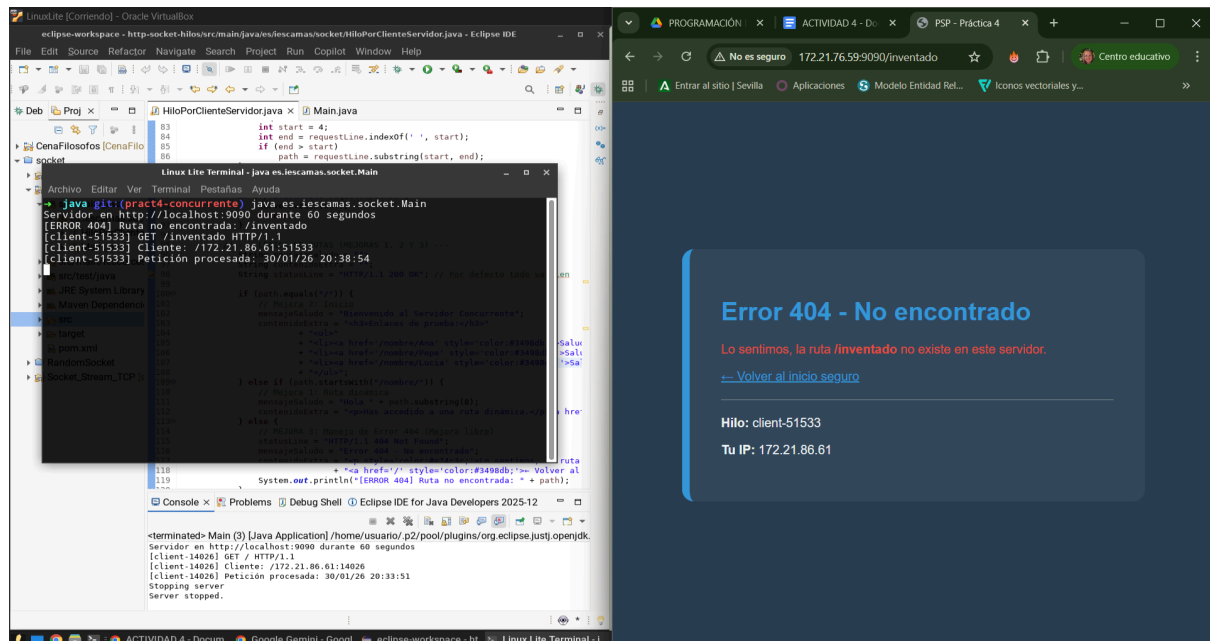
2. Por qué se cambió

- Se implementó para facilitar la navegación. En lugar de obligar al usuario a escribir rutas complejas en la barra de direcciones, el servidor ofrece un menú visual que permite probar la funcionalidad con un solo clic.

3. Cómo comprobar que funciona

1. **Lanzamiento** → Se arranca el servidor desde la terminal de la VM Linux ejecutando la clase Main. El servidor quedará a la escucha en el puerto 9090 durante 60 segundos.
2. **Conexión Inicial** → Desde el navegador del Host, se accede a la IP base: <http://172.21.76.59:9090>.
3. **Resultado esperado** → Se observa una interfaz con el título "Bienvenido al Servidor Concurrente" y una sección de "Enlaces de prueba" con accesos directos a Ana, Pepe y Lucia.
4. **Verificación** → En la terminal de la VM Linux, debe aparecer el registro confirmando la ruta accedida:
 - a. [client-46720] GET / HTTP/1.1
 - b. [client-46720] Cliente: / 172.21.86.61:46720
 - c. [client-46720] Petición procesada: 30/01/26 20:12:13

Mejora 3: Manejo de Errores 404 Not Found



1. Qué se cambió

- **Servidor (HiloPorClienteServidor.java)** → Se implementó un **else** final en la lógica de ruta para capturar cualquier petición a una dirección no definida.

2. Por qué se cambió

- Para asignar al servidor de un comportamiento profesional ante entradas erróneas. En lugar de fallar o devolver una página vacía, el servidor informa al usuario exactamente de qué ha fallado, mejorando la seguridad del servicio.
- Permite al administrador monitorizar intentos de acceso a rutas inexistentes. Cada vez que ocurre un error 404, el servidor imprime un aviso en la consola de Linux detallando la ruta fallida.

3. Cómo comprobar que funciona

1. **Lanzamiento** → Se arranca el servidor desde la terminal de la VM Linux ejecutando la clase Main. El servidor quedará a la escucha en el puerto 9090 durante 60 segundos.
2. **Error forzado** → Desde el navegador del Host, se intenta acceder a una ruta inexistente, por ejemplo: <http://172.21.76.59:9090/inventado>.

3. **Resultado esperado** → El navegador muestra el mensaje "Error 404 - No encontrado" y especifica que la ruta /inventado no existe.
4. **Verificación en Consola** → En la terminal de la VM Linux, debe aparecer el registro confirmando la ruta accedida:
 - a. [client-51533] GET /inventado HTTP/1.1
 - b. [client-51533] Cliente: / 172.21.86.61:51533
 - c. [client-51533] Petición procesada: 30/01/26 20:38:54

? Preguntas de comprensión

1. ¿Qué línea exacta crea el hilo por cliente? Pégala y explícalo en 2-3 frases.

```
new Thread(() -> {  
    try {  
        processClientRequest(clientSocket);  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}, "client-" + clientSocket.getPort()).start();
```

- Crea un nuevo objeto `Thread` al que se le pasa una tarea (procesar la petición del cliente) mediante una expresión lambda. Al ejecutar el método `.start()`, el servidor abre un flujo de trabajo independiente para ese cliente, permitiendo que el hilo principal quede libre inmediatamente para aceptar la siguiente conexión.

2. ¿Qué pasaría si no se crearan hilos y se llamara directamente a `processClientRequest()`?

- El servidor se volvería secuencial o iterativo. Esto significa que solo podría atender a un cliente a la vez; si un cliente realizara una petición que tardara en procesarse, todos los demás usuarios que intenten conectarse quedarían bloqueados en espera hasta que el anterior termine. No habría multitarea y la eficiencia del servidor caería drásticamente.

3. ¿Qué evidencias aportas para demostrar concurrencia real?

- La prueba principal es el campo **"Hilo encargado"** que aparece en el navegador. Al abrir varias pestañas o conectarse desde distintos dispositivos, se puede observar que cada petición es atendida por un hilo con un nombre diferente (por ejemplo, client-53724 y client-53725). Además, en la consola de la VM se puede ver cómo el servidor registra múltiples peticiones procesadas en el mismo segundo por hilos distintos.

4. ¿Por qué usamos VM + host? ¿Qué demuestra ver IPs distintas?

- Usamos VM + host para simular un entorno de red real. Al ver que la IP del servidor es la de la VM (Linux) y la IP del cliente es la del Host (Windows), demostramos que la comunicación no es solo interna (localhost), sino que el socket está escuchando correctamente en la red y es capaz de gestionar conexiones externas que viajan entre diferentes sistemas operativos.

5. Explica tus dos mejoras libres: qué/por qué/cómo comprobar.

- Mejora 2: Panel de Inicio y Diseño Visual

- **Qué/Por qué** → Se creó una página principal (/) con un menú de enlaces y estilos CSS. Se hizo para que sea más útil, permitiendo probar el servidor con clics en lugar de escribir URLs, y para presentar la información técnica de forma más profesional y legible.
- **Cómo comprobar** → Accediendo a la IP del servidor sin rutas adicionales; aparecerá un panel oscuro con botones funcionales hacia diferentes saludos.

- Mejora 3: Manejo de Errores 404 Not Found

- **Qué/Por qué** → Se programó una respuesta específica para rutas que no existen. Se hizo para dar mayor fiabilidad al servidor, cumpliendo con el protocolo HTTP y evitando que la conexión se corte sin dar ninguna explicación o que aparezca una página vacía.
- **Cómo comprobar** → Escribiendo cualquier ruta falsa (ej: /test) en el navegador; el servidor mostrará un mensaje de "Error 404 - No encontrado" y dejará constancia en los logs de la consola.